Using HPC to speed-up the co-clustering algorithm

Ivar van der Spoel s3167445

May 2025

1 Introduction

For this assignment, I will optimize the co-clustering algorithm using MPI[Sni98], CUDA[SK10], OpenMP[DM98] and kernel-tuner[vW19]. This will be ran on the DAS6 cluster computer. To minimize the impact of Amdahl's Law[Gus88]. I will conduct tests on the larger datasets: xl-3x20 and xxl-5x100 to ensure adequate scalability. The time measured is 'total execution time', which will include the time taken to load in the matrix and write the labels.

Small disclaimer: After benchmarking I found a big speed-up in `update_col_labels` by putting a break as seen in Figure 1, but sections 2 and 3 do **not** have this in their benchmarks, and re-doing the benchmarks was not possible due to DAS6 being busy. So when testing it on your own it can be even faster!

```
    dist += calculate_distance(y, item);
    if (dist > best_dist)
        break;
}
if (dist < best_dist) {
    best_dist = dist;
    best_label = k;
}
```

Figure 1: Break to speed-up update function.

2 MPI

Working with multiple computers (or rather, Nodes), we can distribute the work to boost performance.

2.1 Implementation

This implementation leverages MPI to distribute computation across multiple nodes, enhancing performance by parallelizing work and coordinating communication. The matrix is split so each node processes a subset of rows and columns.

1. Calculate Cluster Average Each node calculates local sums and counts of matrix elements grouped by cluster labels on its assigned data. These partial results are combined across all nodes using MPI_Reduce. The root node computes the final cluster averages and broadcasts them to all nodes with MPI_Bcast, ensuring all nodes work with consistent data.

2. Update Row Labels Nodes compute distances from their assigned rows to all possible row labels based on local data in a local distance matrix. The distance information is reduced to the root node, which selects the best labels and tracks how many labels change. The updated row labels are then broadcast back to all nodes.

3. Update Column Labels Each node updates column labels for its assigned subset of columns independently by minimizing distance to cluster averages. Since each node works on disjoint columns, this step requires no MPI communication.

2.2 Benchmarking

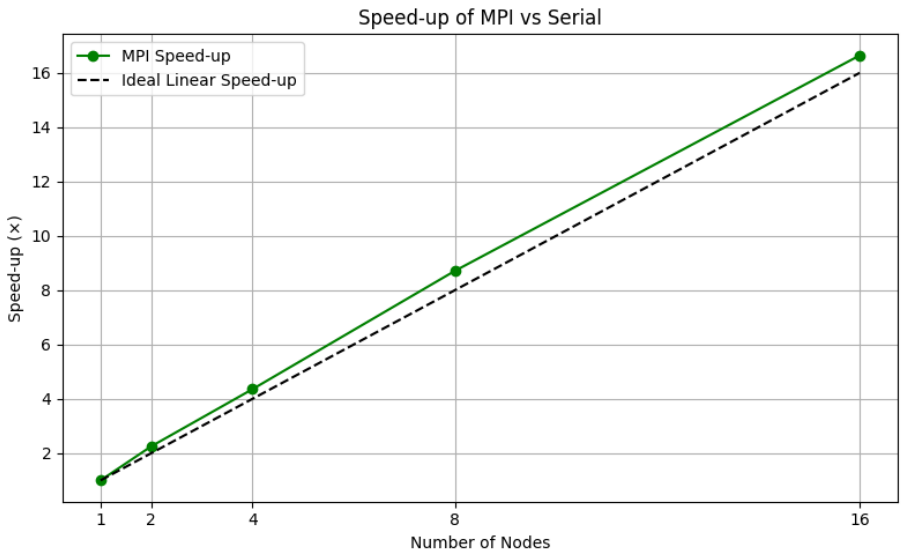

Figure 2: MPI speed-up on XL-3x20.

The speedup results of my MPI implementation can be found in Figure 2. The fact that XL-3x20 takes more than 15 minutes on 1 node is more than logical. 1 Node MPI is essentially serial, and according to the assignment it takes 18:34 minutes on average to complete this dataset. The speed-up is almost linear for MPI, which means that there is a good distribution of load balancing.

3 Cuda

GPU kernels can potentially be a significant speed-up as well.

3.1 Implementation

Alongside MPI, each of the three functions now includes a kernel which utilizes the GPU to increase performance. As we are only iterating over columns we also only load in the assigned columns into each GPU.

1. Calculate Cluster Average Each thread does 1 cell of the matrix, gets the labels and atomically adds the value to the cluster sum and increments the cluster size. We do this atomically, because multiple threads want to increment in the same cell which could cause race conditions. After the kernel it stays the same as in MPI where we reduce, do the averaging on the head node and then broadcast this data.

2. Update Row Labels This function also includes a local distance matrix which has each row and each row label just like MPI. Now each thread is assigned a column and goes over all row and label combinations, then calculates the distance with that column's label which adds it to the local distance matrix (atomically, just like before due to race conditions). After, we reduce these results and the head node figures out which rows need to be updated.

3. Update Column Labels This function assigns one thread per column. Each thread loops over all rows and tries every possible column label, picking the one that gives the lowest distance based on the current row labels and cluster averages. If the new label is different from the old one, the thread updates the label and atomically increments a counter the counter and distance sizes

3.2 Benchmarking

Dataset	1	2	4	8	16
xl-3x20	1:47	0:59	0:32	0:20	0:14
xxl-5x100	8:24	4:26	2:26	1:15	0:57

Table 1: Cuda implementation on xl-3x20 and xxl-5x100 with different amount of nodes.

The results from the Cuda implementation can be found in Table 1. Unsurprisingly, the Cuda implementation is much faster than when only utilizing MPI.

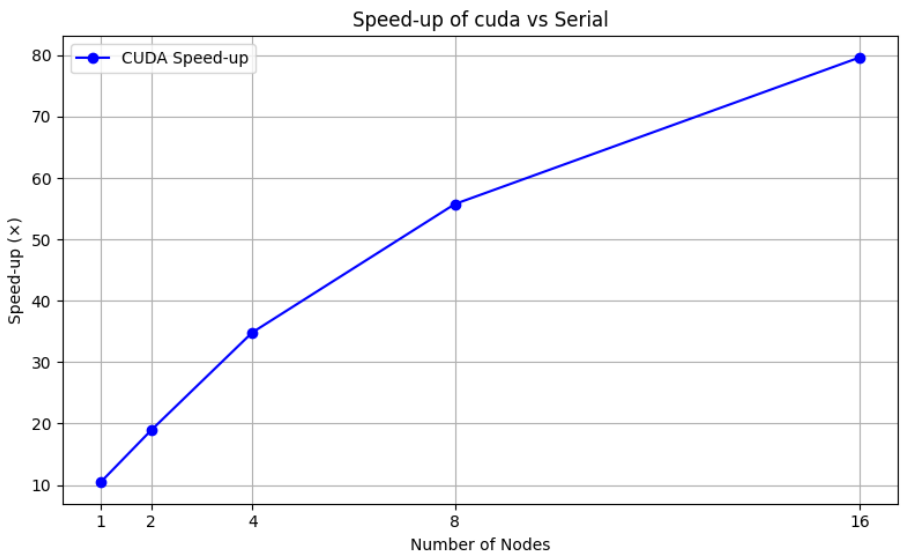

Figure 3: Cuda speed-up on XL-3x20.

In Figure 3 the speed-up of CUDA is shown on XL-3x20. The speed-up is significant as we hope. The fact that it is not linear is because the majority of the time is spent not from doing the actual calculations of the algorithm, but overhead in the form of loading in the data and writing the labels, these are equally fast on both processes.

3.3 Comparing sequential with cuda

In Table 2 we compare the serial implementation to that of the cuda implementation. It becomes obvious that utilizing the Nvidia GPU we can get a much better performance, with it being **37**

times faster on XXL-5x100!

Table 2: Runtimes (in seconds) for Serial and CUDA implementations across dataset sizes on 1 node.

Labels	**S**	**M**	**L**	**XL**	**XXL**
3×20 (Serial)	3	14	125	1115	4496
3×20 (CUDA)	1	2	13	107	430
5×100 (Serial)	8	56	507	4600	18590
5×100 (CUDA)	1	3	15	124	504

4 Additional points

I implemented more features to optimize the program.

4.1 Optimizing `update_row_label`

Instead of iterating over rows, we iterate over columns.

4.1.1 Implementation

There are only 40 rows and up to millions of columns to go over in this function. When having multiple nodes we would like to distribute the work evenly, and when 1 node has to go over one more row than another node, this could end up being millions of matrix values more. Therefore, a better way to split up the work is to do it column-based. This caused a speed-up. However, this could also cause cache misses. This could be resolved by having a transposed matrix, I did not implement this.

4.1.2 Benchmarking

Program	**1**	**2**	**4**	**8**	**7**
Column-based	>15:00	8:15	4:16	2:08	2:10
Row-based	>15:00	8:20	4:19	2:10	2:31

Table 3: Two versions of MPI on xl-3x20.

The rows can be nicely split up with 2 4 and 8 columns, which means there are no rows that are distributed poorly, each node has an equal amount of rows. However when the rows cannot be evenly distributed, for example when there's 7 nodes, the rows cannot be evenly split causing a significant slow down as seen in Table 3.

4.2 Using OpenMP with MPI application

Another improvement can be done by utilizing OpenMP, this will use multiple cores on a CPU.

4.2.1 Implementation

Each of the three function has for-loops. The `calculate_cluster_average` function is parallelized by each thread using local arrays to accumulate results independently, avoiding race conditions. The nested loops over rows and columns are collapsed with for better load balancing. After local computation, results are merged into shared arrays inside a critical section to ensure thread safety. In `update_row_label` there is a collapse when calculating the aforementioned local distance. We can also reduce the total distance accumulation. For `update_col_labels` the same is present, we can reduce the total distance calculations.

4.2.2 Benchmarking

The results of OpenMP can be found in Table 4. There is a significant speed-up by using OpenMP. However, the amount of cores do not affect the program. I do not understand what is happening. I checked the performance of my old program (Control), implementation of OpenMP and then adjusting the cores.

Cores	**l-5x100**
Control	5:03
1	4:44
2	3:48
8	3:48
16	3:48
32	3:48
64	3:48

Table 4: Comparing OpenMP core variations with 1 node on l-5x100.

4.3 Shared memory, warp shuffle reduction and atomics

More speed-ups in kernels can be done by utilizing shared memory, warp shuffle reductions and atomic operations.

4.3.1 Implementation

Warp shuffles only makes sense in `update_row_labels`, because we want to iterate over all the columns, not rows. Each warp processes 1 pair of potential row label with a matrix row. They make jumps of 32 where each thread on a warp does the one next to each other. Each thread accumulates all the distances and then they're reduced together, and then are added to the local distance matrix. As the size of shared memory is limited, I only put row_labels in it (40 values) and the cluster averages.

4.3.2 Benchmarking

The results can be seen in Table 5. Shared memory seemed to make the performance consistently worse. Shuffles improved the performance a lot.

Implementation	**1**	**2**	**4**	**8**	**16**
Shuffles	3:32	2:07	1:15	0:49	0:37
Shuffles + Shared	3:34	2:09	1:18	0:51	0:39
None	8:24	4:26	2:26	1:15	0:57

Table 5: Measuring xxl-5x100 with different node counts on implementations with shuffles, shuffles + shared memory and neither.

4.4 Mixed precision

Doubles are twice as many bits than floats, and may be redundant!

4.4.1 Implementation

I implemented this by making distance accumulations in `update_row_labels` and `update_col_labels` floats instead of doubles, and the cluster average a vector are now floats instead of doubles. We have to make sure the broadcasting and reductions are done correctly. The total global distance still remains a double, as well as the accumulated global summation of the matrix values (but when calculating the averages, they are converted to floats).

4.4.2 Benchmarking

The results can be found in Table 6. This causes a significant speedup. The NMI still remains at 99.995%! This suggests that we could maybe increase it to float16s or float8s.

Implementation	**1**	**2**	**4**	**8**	**16**
w/o Mixed Precision	3:32	2:07	1:15	0:49	0:37
w/ Mixed Precision	1:49	1:12	0:44	0:33	0:30

Table 6: Measuring xxl-5x100 on different node counts with and without mixed precision.

4.5 Kernel Tuner to auto-tune kernel configurations

Better dimensions in my kernels should make it even faster.

4.5.1 Implementation

I used Ben van Werkhoven's Kernel Tuner to auto-tune my kernel configurations. I used the google colabs as a reference point on how to write my own auto-tuner. I scrape the data from the files and write my own .cu files, then I tune them. I also make sure to mimic the input data realistically. You can do it yourself in my run_tuner job file where the last command should be:

```
/usr/bin/python3.12 ./tuner.py 1-16 s|m|l|xl|xxl 3x20|5x100
```

This will tune the kernels according to the amount of nodes, files and label sizes respectively.

4.5.2 Benchmarking

To test if there are any improvements, I ran my auto-tuner for 8 nodes on xxl-5x100, as seen in the previous assignment the goal is to beat 0:33. The results of the auto-tuner can be found in Tables 7, 8 and 9. The best configurations are 32x32, 32 and 256 respectively. So, I updated my cgc_cuda.cu file and reran xxl-5x100 with 8 nodes. Instead of **0:33** I got **0:32** consistenly over multiple runs!

5 Discussion

Some measurements were unfortunately not the best due to the new 'break' statement as mentioned in the disclaimer. Due to a high competitive occupation of the DAS there was no averaging over all the outputs. There are many different things to implement and test, such as matrix transposition to reduce cache misses, combining the kernels and for each node to only load in the necessary part of the matrix rather than loading in the entire matrix and then splitting it up. I could also use the NVidia GPU analyser to observe what happens within the GPUs better. Streams and Asynchronous calculations would be another potential speedup alongside many other things.

block_size_x	block_size_y	time (ms)
16	1	61.911
16	2	49.273
16	4	43.312
16	8	38.573
16	16	33.489
16	32	33.527
32	1	52.768
32	2	40.131
32	4	35.057
32	8	31.584
32	16	26.140
32	32	**25.647**
64	1	52.720
64	2	40.139
64	4	35.057
64	8	31.558
64	16	26.122
128	1	52.704
128	2	40.116
128	4	35.041
128	8	31.552

Table 7: Auto-tuning `calculate_cluster_average`.

block_size_x	time (ms)
16	67.086
32	**44.577**
64	45.981
128	86.882
256	175.241

Table 8: Auto-tuning `update_row_labels`.

block_size_x	time (ms)
16	0.261
32	0.153
64	0.086
128	0.059
256	**0.044**

Table 9: Auto-tuning `update_col_labels`.

6 Conclusions

Overall I am happy with my implementation. There is a big speedup with MPI and even bigger with Cuda. I learned working with mixed-precision, Kernel tuner, MPI, OpenMP, writing kernels, using shared memory and warp shuffles and I would definitely like to do more HPC!

References

[DM98] Leonardo Dagum and Ramesh Menon. Openmp: an industry standard api for shared-memory programming. *IEEE computational science and engineering*, 5(1):46–55, 1998.

[Gus88] John L Gustafson. Reevaluating amdahl's law. *Communications of the ACM*, 31(5):532–533, 1988.

[SK10] Jason Sanders and Edward Kandrot. *CUDA by example: an introduction to general-purpose GPU programming*. Addison-Wesley Professional, 2010.

[Sni98] Marc Snir. *MPI–the Complete Reference: the MPI core*, volume 1. MIT press, 1998.

[vW19] Ben van Werkhoven. Kernel tuner: A search-optimizing gpu code auto-tuner. *Future Generation Computer Systems*, 90:347–358, 2019.